



University
of Regina

Go far, *Together.*

ENSE 375 – Software Testing and Validation

Personal Password Manager

Korbin Wyssen (200482024)

Poojitha Dayalan (200470748)

Adai Yisah (200365751)

Table of Contents

1	Introduction.....	5
2	Design Problem.....	6
2.1	Problem Definition.....	6
2.2	Design Requirements.....	6
2.2.1	Functions.....	6
2.2.2	Objectives.....	7
2.2.3	Constraints.....	8
3	Solution.....	12
3.1	Solution 1.....	12
3.2	Solution 2.....	12
3.3	Final Solution.....	12
3.3.1	Components.....	12
3.3.2	Environmental, Societal, Safety, and Economic Considerations.....	12
3.3.3	Test Cases and results.....	12
3.3.4	Limitations.....	13
4	Team Work.....	13
4.1	Meeting 1.....	13
4.2	Meeting 2.....	13
4.3	Meeting 3.....	14
4.4	Meeting 4.....	14
5	Project Management.....	15
6	Conclusion and Future Work.....	16
7	References.....	17
8	Appendix.....	18

List of Figures

List of Tables

1 Introduction

This project aims to relieve the pain experienced by users having to create and manage multiple accounts' passwords by offering a program the user can use to securely store their various passwords for these accounts that online services require. The program not just gives the user with a way to keep track of both the user's password to a website, but also allows the user to setup 2FA for their account through the use of a Time-based One-Time Password (TOTP), as well as keeping a record of the account's recovery keys in the event the user has difficulties logging into their 2FA secured account.

2 Design Problem

2.1 Problem Definition

In this age of online services and web applications, many websites require that users create a personal account so that they can access the features and services the application offers. Majority of applications require a user's account to be secured with a text-based password, and some method of two-factor-authentication, to ensure that only the authorized user who created the account can log into the account.

This creates a strain on the user to memorize different passwords for each account and be able to recall what password was used for which account. Many users opt to either use the same password for multiple accounts or utilize dictionary to both meet general password length requirements and make the password rememberable.

These both create security issues for the user, as if the user's password hash is leaked due a site's security being compromised, and is cracked, it could be used to gain access to a user's other account on another site. Furthermore, dictionary-based passwords are easy for an infiltrator to brute-force and crack.

2.2 Design Requirements

2.2.1 Functions

1. **Store Passwords:** The solution must be able to store user login information for various websites in the form of login entries.
2. **Store Text-based Passwords:** A text-based login entry must be able to store a password.
3. **Store TOTP-based Keys:** A TOTP-based login entry must be able to store a TOTP security key.
4. **Store Recovery Keys:** A TOTP-based login entry must be able to store associated account recovery keys.
5. **Base32 decoding:** The solution must have base32 decoding abilities for security keys.

6. **Generate one-time password:** The solution must be able to generate correct one-time passwords from the security key of a given login entry.
7. **HOTP specification:** The solution must use the HMAC-one-time-password specification to generate correct one-time passwords.
8. **One-time password validity:** The solution should inform the user how long the generated one-time password is valid for.
9. **List login entries:** The solution should have the ability to list all login entries to the user.
10. **Filter login entries:** The user should have the ability to search or filter through the list of login entries
11. **Create login entry:** The user must be able to create a new login entry.
12. **Modify login entry:** The user must be able to modify an existing login entry.
13. **Delete login entry:** The user must be able to delete an existing login entry.
14. **Create database file:** The solution must be able to create a new database file in the instance an existing database file could not be found.
15. **Read database file:** The solution must be able to read the database file.
16. **Write database file:** The solution must be able to write to the database file.
17. **Encrypt data:** The solution must have the ability to encrypt a user's login entries before writing to the database file.
18. **Decrypt data:** The solution must have the ability to successfully decrypt a user's login entries back to the original data after reading the database file.
19. **Prompt for password:** The solution must be able to prompt the user for a primary key to decrypt the database file.

2.2.2 Objectives

1. Develop a functional local desktop password manager that provides centralized management of account credentials.

2. Provide secure local storage for usernames, password, time-based one-time password (TOTP) information, and recovery keys.
3. Reduce the user's dependence on memorizing multiple account passwords by providing controlled access to stored credentials.
4. Protect stored credentials from unauthorized access through authentications and encryptions.
5. Provide the core credential-management functions required to add, retrieve, search, modify, and delete stored account information.
6. Develop the application using an iterative engineering design process in which alternative designs, algorithms, modules, and feature additions can be evaluated before the final design is selected.
7. Produce a design that is modular, testable, and suitable for systematic validation using the testing required by ENSE 375.

2.2.3 Constraints

Economic Factors:

Constraint: As students, we have limited budgets and cannot rely on paid cloud services, API or any other hosting infrastructure. Most of the security tools are expensive to use, and it is financially out of our reach, still the system requires strong security.

Solution: Using free and open-source cryptographic libraries and storing all the data locally on the user's device. Using free developer tools like GitHub, JUnit and VS code. Verify before using any library which is free for development and safe to use.

- Finance and budget limitations
- Use free and open-source tools and libraries
- Only use the user's local system resources (storage and computational power); Do not use cloud services to off-load resource requirements
- Only use the Java Programming Language for implementation of solution logic

Regulatory Compliance (Security and Standards):

Constraint: This application handles sensitive information, and there are many major issues like password leaks, weak passwords, and accidental data release. Security failures will lead to serious safety and credibility issues.

Solution: Users will have to enter a master password before accessing other stored credentials. All sensitive information, like passwords, should be protected from external users. Security requirements will be verified through appropriate integration and validation testing.

- Use academia/course approved tools (git, GitHub, VSCode, Eclipse IDE)
- Use JUnit for requirements implementation testing
- Use the test-driven development methodology for implementation

Reliability:

Constraint: The password manager application might crash while saving the passwords and make the system unavailable for a while. Users might lose access to their accounts while incorrectly modifying stored credentials.

Solution: Password managers will use storage to enter and manage more data, and it reduces the possibility of corruption and data loss. Adding, Modifying, retrieving and deleting credentials must be tested through Integration and unit testing methods.

- The database file is reliant on the condition of the physical storage medium it's stored on.
- Cannot automatically create external backups of the database file.
- The solution must rely on the user's ability to remember the primary password to unlock the database file.

- The solution must rely on the user's initiative to ensure login entries are kept up to date.

Ethics:

Constraint: Password manager manages private and sensitive information. There are many risks in the real world. We have an ethical responsibility to avoid revealing, misusing, or collecting other user's info.

Solution: The development team should focus on Confidentiality, Privacy and responsible for handling user's information.

- By the sensitive nature of login details and passwords, the solution must encrypt data in the database file to uphold security expectations from the user.

Architectural Constraints:

We are using MVC architecture for this project. It follows the principle of "Separation of concerns". It organizes the code in an efficient way by separating it into different parts based on its purpose rather than mixing all the responsibilities.

Model: This part of code deals with the data of the application

View: This part of code deals with the UI of the application

Controller: This part of code deals with the functionality of the application

Testing Constraints:

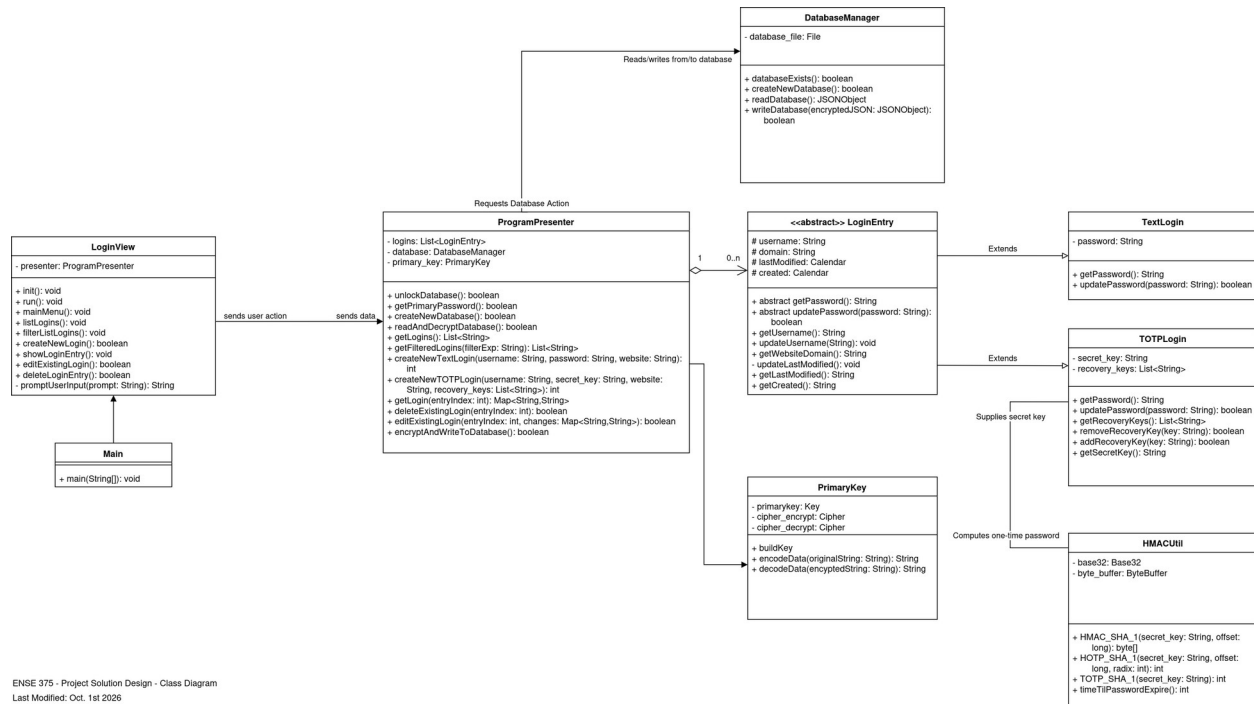
Testing requirements create several testing constraints while building the project. The following testing techniques must be used in this project.

- **Test Driven Development:** Testing must be considered while developing the project rather than waiting for the project to be completed.
- **JUnit Testing:** JUnit testing needs to be done for most of the unit test cases.

- **Integration Testing:** The whole application needs not to be tested as a single unit. A subset of units must be selected.
- **Path Testing:** Based on the possible execution path, at least one function should be tested.
- **Data flow Testing:** At least one function should be tested on how the data is defined, modified, and used in the project.
- **Boundary Value Testing:** Input values at important boundaries should be tested.
- **Equivalence class testing:** Input values will be separated into valid and invalid classes, and it needs to be tested.
- **Decision tables testing:** Decision tables testing will be created to test the functionality that depends on many conditions.
- **State transition testing:** The testing will be performed in the application based on different states and transitions.
- **Use case testing:** This testing will be performed based on the user's perspective.

3 Solution

3.1 Solution 1

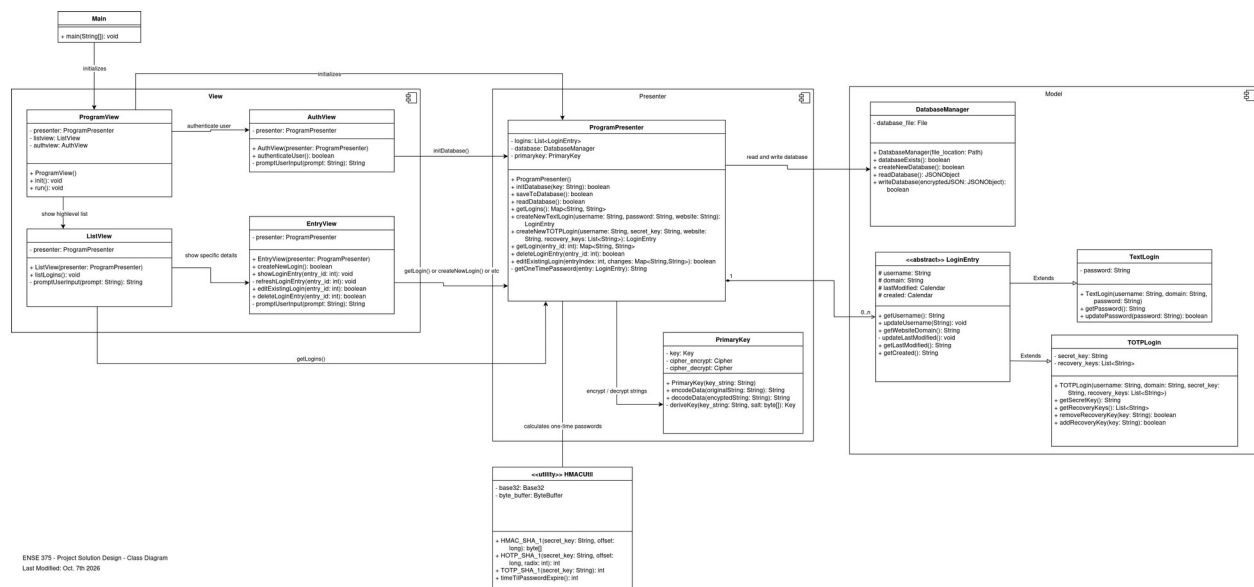


Our first solution was designed with the MVP (Model-View-Presenter) architecture. Initially, we have a single **LoginView** class that handles all user interactions including the main menu, creating and deleting entries, listing entries, and prompting input. A **ProgrammingController** was placed between the View and the model. It unlocks the database, derives the primary key, created and edited logins, and generates TOTP (Time-Based One-Time Generator). In the model, it has a **DatabaseManager** for file storage, **Primary key** is used for encryption and decryption. There are also other abstract classes called **LoginEntry** with **TextLogin** and **TOTPLogin** as subclasses. **HMACUtil** for TOTP Calculation.

We did not select the design due to this testability. **ProgramController** has too many responsibilities including handling databases access, TOTP generation and entry management and authentication. **LoginView** combines all the mainview, login entries and all kinds of user inputs cannot be tested alone. Testing any of

them needs to be set up the other and it would be large and the testing is dependent on the other. The controller and the view depend on each other, and we couldn't test the controller logic as there is no real interface.

3.2 Solution 2



Our solution 2 is the improved solution of Solution 1. It was also designed based on the MVP architecture. Here the **View** is split into four classes: **ProgramView**, **AuthView**, **ListView**, **EntryView**. Each of them handles every part of the user flow which includes Initiating the application, authenticating it, Listing and Editing the entries. Each of them refers to the **ProgramPresenter**, which includes initializing the database, editing, saving and reading login data, create and edit logins and requesting and generating TOTP. **HMACUtil** was moved from the **TOTPLogin** class to the **ProgramPresenter** class to put additional responsibility on the **ProgramPresenter** for translating data structures between the model and the view, and to reduce having unencrypted data stored in memory for extended periods of time. Only the **ProgramPresenter** communicates with the **DatabaseManager**, **HMACUtil**, **PrimaryKey** and **LoginEntry** classes.

This design can be easily tested for various reasons, such as all the Views can be tested with a mock presenter. Each view is small, so it can be checked alone to confirm whether it calls the right presenter method. All the components must be tested independently. **Primary Key** can be checked by encrypting and decrypting the known strings. **HMACUtil** function can be tested and verified by known TOTP test vectors. Presenter logic can be tested without any UI. The presenter never handles any input or output directly and can be tested by function calling methods. The user input is also isolated and it is easy to test. All these properties let us test each component by itself before even implementing them, So, it seems to be a better and improved version of Solution 1.

3.3 Final Solution

This is the final solution. **Explain why it is better than other solutions** (focus more on testing). You may use a table for comparison purposes. After providing the reason for selecting this solution, detail it below.

3.3.1 Components

What components you used in the solution? What is the main purpose of using individual component? What testing method did you employ for each component? Provide a block diagram (with a numbered caption, such as Fig. 1) representing the connectivity and interaction between all the components.

3.3.2 Environmental, Societal, Safety, and Economic Considerations

Explain how your engineering design took into account environmental, societal, economic and other constraints into consideration. It may include how your design has positive contributions to the environment and society? What type of economic decisions you made? How did you make sure that the design is reliable and safe to use?

3.3.3 Test Cases and results

What test suits did you design to test your prototype? How did you execute the test cases to test the prototype?

3.3.4 Limitations

Every product has some limitations, and so is the case with your design product. Highlight some of the limitations of your solution here.

4 Team Work

4.1 Meeting 1

Time: Thurs. Sept. 10th 2026, 12:45-12:55pm

Agenda: Delegate work, have work completed by Wednesday

Team Member	Previous Task	Completion State	Next Task
Korbin Wyssen	N/A	N/A	Work on project business case docx
Poojitha Dayalan	N/A	N/A	Assist on project business case
Adai Yisah	N/A	N/A	Assist on project business case

4.2 Meeting 2

Time: Thurs. Sept. 17th 2026, 12:45-12:55pm

Agenda: Review current task progress. Delegate tasks for next part

Team Member	Previous Task	Completion State	Next Task
Korbin Wyssen	Work on project business case docx	100%	Section 2.2.1 Functions
Poojitha Dayalan	Assist on project business case	100%	Section 2.2.3 Constraints
Adai Yisah	Assist on project business case	Missed	Section 2.2.3 Objectives

4.3 Meeting 3

Time: Wed. Sept. 23rd 2026 10:00am-10:30am

Agenda: Update on task progress, finalized section 2.2 and sub-sections

Team Member	Previous Task	Completion State	Next Task
Korbin Wyssen	Section 2.2.1 Functions	100%	TBD
Poojitha Dayalan	Section 2.2.3 Constraints	100%	TBD
Adai Yisah	Section 2.2.3 Objectives	100%	TBD

4.4 Meeting 4

Provide a similar description here.

5 Project Management

Provide a Gantt chart showing the progress of your work here. Mention all the tasks along with their predecessors. Provide the slack time of each task and identify the critical path.

6 Conclusion and Future Work

- A summary of what you achieved. Mention all the design functions and objectives that you achieved while satisfying testing requirements?
- While keeping the limitations of your solution, provide recommendations for future design improvements.

7 References

- Use the IEEE reference style.
- Do not put any reference if it is not cited in the text.

8 Appendix

If you want to provide an additional information, use this appendix.